

Lab3 中断

一、练习1：完善中断处理

1. 实现过程(trap.s的完整流程)

首先,设置中断入口:

在init.c中调用 `idt_init()` (在内存初始化前后分别调用,保证内存初始化后中断地址绑定正确的虚拟地址).

- 首先通过 `write_csr(sscratch, 0)` 表明当前处于内核态;
- 然后使用 `write_csr(stvec, &__alltraps)` 设置异常向量基地址寄存器 (`stvec`), 指向汇编入口函数 `__alltraps`;
- 当中断或异常发生时, CPU 会自动跳转到 `__alltraps` 执行.

这一函数确定了中断的通义入口(`__alltraps`),再由trap.c分发处理.

其次,汇编入口`__alltraps`:

此函数在trapentry.s中,

- 保存现场 (即 CPU 各寄存器状态);
- 构造一个 `trapframe` 栈帧结构;
- 调用 C 函数 `trap()` 进行进一步分析与分发;
- 最后从 `__trapret` 恢复现场并执行 `sret` 返回.

通过汇编部分的配合,实现了“从硬件中断信号”到“内核C语言函数”的过渡.

然后,进行**中断分发**:

在 `trap()` 函数中,通过 `trap_dispatch()` 判断 `tf->cause` 的符号位:

- 若为负,则为中断;
- 若为正,则为异常.

中断交由 `interrupt_handler()` 处理,异常交由 `exception_handler()` 处理.

最后是,**中断和异常的处理**

这是本次实验的重点内容,主要进行时钟中断的处理,并输出 `100 ticks`.

2. 定时器中断及处理过程

当时钟中断发生时,硬件触发中断并通过SBI进入内核;CPU根据stvec寄存器中的地址跳转执行__alltraps函数,在此保存所有寄存器状态并构造trapframe;随后控制流转入C语言层的trap()函数,由trap_dispatch()判断中断类型为IRQ_S_TIMER;接着interrupt_handler()执行相应逻辑,调用clock_set_next_event()设置下一次时钟中断,更新计数器并输出状态信息,在达到条件时触发自动关机;处理完成后,__trapret通过RESTORE_ALL恢复现场,最后由sret指令返回至中断发生前的执行点,至此实现了从硬件触发到正常程序恢复的完整处理流程。

3. 代码逻辑

在实现中,在 `trap.c` 中定义了全局静态变量 `ticks` 用于记录时钟中断的触发次数。每当系统进入时钟中断分支时,就对该计数器加一。当 `ticks` 达到 100 的倍数时,调用 `print_ticks()` 输出提示信息“100 ticks”;当输出达到 10 次(即 `ticks == 1000`)后,通过 `sbi_shutdown()` 接口实现系统关机。该接口定义在 `sbi.h` 中,负责通过 SBI 调用底层硬件服务,触发虚拟机监控层或硬件仿真环境中的关机操作。

时钟中断的触发流程是:硬件定时器到期 → 触发中断请求 → CPU 进入内核态执行 `trap()` → 判断中断类型为 `IRQ_S_TIMER` → 执行时钟中断处理逻辑 → 调用 `clock_set_next_event()` 设置下一次时钟事件 → 恢复现场返回用户态。这样形成一个稳定的周期性中断机制。

以下是 `trap.c` 中完善的核心代码片段:

代码块

```
1      case IRQ_S_TIMER:
2          /* LAB3 EXERCISE1  YOUR CODE : */
3          /*(1)设置下次时钟中断- clock_set_next_event()
4          *(2)计数器 (ticks) 加一
5          *(3)当计数器加到100的时候, 我们会输出一个`100ticks`表示我们触发了100次时
        钟中断, 同时打印次数 (num) 加一
6          * (4)判断打印次数, 当打印次数为10时, 调用<sbi.h>中的关机函数关机
7          */
8          //2312130 景千夏BEGIN
9          /*clock_set_next_event的实现和声明在driver/clock*/
10         clock_set_next_event();
11         tickNum++;
12         if(tickNum%TICK_NUM==0){
13             print_ticks();
14             printCount++;
15         }
16         if(printCount==10){
17             sbi_shutdown();
18         }
19         //cprintf("Supervisor timer interrupt\n");
20         //2312130 景千夏END
21         break;
```


二、扩展练习 Challenge 1：描述与理解中断流程

ucore中处理中断异常的流程

（一）中断概念

在进行具体流程描述之前，我们先阐述中断机制的概念。

中断概念

- 中断是指由于接收到来自外围硬件（相当于中央处理器和内存）的异步信号或来自软件的同步信号，而进行相应的硬件/软件处理。发出这样的信号称为进行中断请求。硬件中断导致处理器通过一个上下文切换来保存执行状态；软件中断则通常作为CPU指令集中的一个指令，以可编程的方式直接指示这种上下文切换，并将处理导向一段中断处理代码。
- 中断是一种使CPU中止正在执行的程序而转去处理特殊事件的操作，这些引起中断的事件称为中断源，它们可能是来自外设的输入输出请求，也可能是计算机的一些异常事故或其他内部原因。

中断的作用

- 并行操作
- 硬件故障报警与处理
- 支持多道程序并发运行，提高计算机系统的运行效率
- 支持实时处理功能

在Lab3中设置了两种中断——时钟中断和断电中断；接下来我们将从这两种中断的角度来进行中断流程的阐述。

（二）异常处理的开启

首先看init.c文件，这个函数是内核初始化的总控函数，这一部分会按照这个文件中的函数为线索进行分析。

代码块

```
1  #include <intr.h>
2  #include <riscv.h>
3
4  /* intr_enable - enable irq interrupt */
5  void intr_enable(void) { set_csr(sstatus, SSTATUS_SIE); }
6
7  /* intr_disable - disable irq interrupt */
8  void intr_disable(void) { clear_csr(sstatus, SSTATUS_SIE); }
```

这一段中 `intr_enable(void)` 函数的作用是打开全局中断开关，允许CPU响应外部中断。使用 `set_csr` 宏将 `sstatus` 寄存器中的 `SSTATUS_SIE` 位置1，而当SIE=1时，CPU在每条指令执行完毕后会检查是否有待处理的中断，如果有就会跳转到中断处理程序。`intr_disable(void)` 函数的作用是关闭全局中断开关，禁止CPU响应外部中断。其中函数内部的具体操作过程是 `intr_enable(void)` 函数的反过程。

（三）中断处理过程

需要明确的一点是CPU对于中断和异常的具体处理机制本质上是完全一致的，即：当CPU收到中断或者异常的信号时，它会暂停执行当前的程序或任务，通过一定的机制跳转到负责处理这个信号的相关处理程序中，在完成对这个信号的处理后再跳回到刚才被打断的程序或任务中。

（1）触发中断

时钟中断的具体流程为：硬件计时器->CPU中断控制器->CPU核心。这个过程是在物理硬件中完成，不在软件代码中，也就是说触发时钟中断的“扳机”是由硬件扣动的。在我们的源代码中，`clock.c`文件所做的是“装弹”（启用中断）和“瞄准”（设置触发时间），而真正的“击发”是硬件在预设时间点自动完成的。

代码块

```
1  void clock_init(void) {
2      // enable timer interrupt in sie
3      set_csr(sie, MIP_STIP);
4      // divided by 500 when using Spike(2MHz)
5      // divided by 100 when using QEMU(10MHz)
6      // timebase = sbi_timebase() / 500;
7      clock_set_next_event();
8
9      // initialize time counter 'ticks' to zero
10     ticks = 0;
11
12     cprintf("++ setup timer interrupts\n");
13 }
14
15 void clock_set_next_event(void) { sbi_set_timer(get_cycles() + timebase); }
16
```

ucore通过 `clock_init()` 和 `clock_set_next_event()` 函数启用中断并设置第一个超时时间点。当CPU的计时器到达该时间点时，硬件会自动产生中断信号，触发中断，在进入S模式进行处理之前，硬件会自动扮演一名忠实的“现场记录员”，为处理程序准备好一份详尽的上下文报告。

之后

- 首先，保存中断发生时的pc值，即程序计数器的值，这个值会被保存在sepc寄存器中。
- 接下来，保存相关的辅助信息。若异常与缺页或访问错误相关，将相关的地址或数据保存到stval寄存器，以便中断处理程序在后续处理中使用。然后，保存并修改中断使能状态。将当前的中断使能状态sstatus.SIE保存到sstatus.SPIE中，并且会将sstatus.SIE清零，从而禁用S模式下的中断。从而保证在处理中断时不会被其他中断打断。
- 然后，保存当前的特权级信息。将当前特权级保存到sstatus.SPP中，并将当前特权级切换到S模式。此时，系统已经进入S模式，准备跳转到中断处理程序。将pc设置为stvec寄存器中的值，并跳转到中断处理程序的入口。

(2) 处理中断

在硬件自动执行上下文保存之后，跳转到中断处理程序的入口，开始了软件处理的步骤。在这一部分，也是先进行上下文保存，会保存所有寄存器的值到栈。

代码块

```
1      .globl __alltraps
2      .align(2)
3  __alltraps:
4      SAVE_ALL
5
6      move a0, sp
7      jal trap
8      # sp should be the same as before "jal trap"
```

在这段代码中，`.globl __alltraps` 告知链接器这是一个外部可见的标签，作为所有陷阱的统一入口地址；`.align(2)` 对齐指令，确保 `__alltraps` 标签处的代码地址是4字节对齐，以满足指令获取的要求，提高执行效率。`__alltraps` 为陷阱入口标签，硬件在发生陷阱/中断后，将控制流强制跳转到这个地址开始执行；`SAVE_ALL` 保存上下文宏，这是一系列汇编指令的宏，负责将CPU的所有通用寄存器的值压入当前的内核栈中。这一块保存的数据称为陷阱帧。`move a0, sp` 设置第一个参数，将当前的栈指针sp的值复制到寄存器a0中。在C语言调用约定中，a0用于传递第一个参数。此时sp指向陷阱帧的起始位置；`jal trap` 跳转至C函数，跳转并链接到C语言编写的函数，也就是跳转到trap.c文件进行逻辑判断和处理。

同时，在trap.c文件中，需要进行一系列的准备操作：

i.准备阶段

在系统初始化时，内核会执行 `idt_init` 函数来配置 RISC-V 处理器，告知它在发生陷阱/中断时应该跳转到哪里。

- **清除暂存器：** `write_csr(sscratch, 0);` 这一步将sscratch设置为0，这是一种约定，告诉汇编入口当前是从内核中发生的中断。

- **设置陷阱向量：** `write_csr(stvec, &__alltraps);` 这是最关键的一步。它将stvec设置为汇编入口函数 `__alltraps` 的地址。这意味着，任何特权级的中断或异常发生时，CPU 硬件都会自动强制跳转到 `__alltraps` 开始执行。

ii.C语言处理及分发阶段

这是trap.c文件中的代码的核心部分，由trap->trap_dispatch->interrupt_handler组成。

1. `trap(struct trapframe *tf)` 这是汇编代码跳转后执行的第一个C语言函数。它的主要工作是调用 `trap_dispatch` 函数，将陷阱帧指针tf传递下去。
2. `trap_dispatch(struct trapframe *tf)` 该函数负责分发陷阱类型：
 - 判断类型：它通过检查tf->cause的最高位来判断是中断还是异常

代码块

```
1  if (((intptr_t)tf->cause < 0)) {
2      // 中断 (最高位为 1)
3      interrupt_handler(tf);
4  } else {
5      // 异常 (最高位为 0)
6      exception_handler(tf);
7  }
```

- 中断：调用interrupt_handler
 - 异常：调用exception_handler
3. `interrupt_handler(struct trapframe *tf)` 该函数负责处理各种类型的中断。它通过 `switch` 语句检查 `tf->cause` 的值来确定具体的中断源。
 - **时钟中断处理 (IRQ_S_TIMER) - 关键逻辑：** 当发生监管者时钟中断 (IRQ_S_TIMER) 时，执行以下关键步骤（即您在代码中填写的逻辑）：
 - i. **设置下次中断：** `clock_set_next_event();`
 - 这是周期性时钟中断的必需操作。必须在当前中断中设置下一次中断的时间，否则时钟中断只会发生一次。
 - ii. **增加计数器：** `if (++ticks % TICK_NUM == 0)`
 - 全局计数器 `ticks` 增加 1。
 - 当 `ticks` 达到 `TICK_NUM` (即 100) 的倍数时，执行下一步。
 - iii. **打印和计数：** `print_ticks(); num++;`
 - 打印 `100 ticks`。

- 打印次数计数器 `num` 增加 1。

iv. 关机检查: `if (num >= 10) { sbi_shutdown(); }`

- 当 `num` 达到 10 次时（即发生了 1000 次时钟中断），调用 **SBI (Supervisor Binary Interface)** 函数执行关机操作。
- **其他中断**：对其他中断（如软中断、外部中断等）只进行简单的打印提示或留空。

(3) 中断服务程序完成后恢复现场

在 `trap.c` 处理完中断之后，需要恢复 CPU 之前保存的状态，包括程序计数器、寄存器值等。其对应着 `trapentry.S` 文件中以下部分的代码内容：

代码块

```
1      .globl __trapret
2  __trapret:
3      RESTORE_ALL
4      # return from supervisor call
5      sret
```

这一部分执行最终的上下文恢复和特权级切换，使 CPU 能够安全、无缝地返回到之前被中断的程序继续执行。`.globl __trapret` 通过这一句命令可以告知链接器和 C 语言代码，`__trapret` 是一个可以从外部访问的入口点。C 语言的陷阱处理函数在完成任务后，会明确地跳转到这里。`__trapret` 这个汇编代码中恢复和返回流程的起始点。`RESTORE_ALL` 这是一个汇编宏，用于恢复上下文，它执行一系列指令，从当前栈帧中取出所有之前用 `SAVE_ALL` 保存的通用寄存器、程序计数器等，并将其写回 CPU 寄存器中。这会将 CPU 的状态恢复到中断发生时的样子。`sret` 为特权级返回指令，这个指令是 RISC-V 架构中的关键指令，执行以下操作：

1. 恢复 pc：将程序计数器设置为陷阱帧中保存的返回地址。
2. 切换特权级：将 CPU 从当前的内核模式切换回中断发生时的模式。
3. 恢复状态：恢复其他相关的控制寄存器。

问题回答

`mov a0, sp` 的目的是什么

其目的为：将陷阱帧的地址作为参数传递给 C 语言的陷阱处理函数 `trap`。

- **陷阱帧**：在 `SAVE_ALL` 宏执行之后，所有的 CPU 寄存器（包括通用寄存器、返回地址等）都被按顺序压入了当前的内核栈中。这一整块数据结构就称为“陷阱帧”或“上下文”。
- **栈指针 `sp`**：此时的栈指针 `sp` 正好指向这个新创建的**陷阱帧的起始位置**。
- **寄存器 `a0`**：在 RISC-V 架构中 C 语言函数调用约定中，`a0` 是用于传递**第一个函数参数**的寄存器。

- **桥接 C 语言：**当执行 `jal trap`（跳转并链接到 C 函数 `trap`）时，C 函数 `trap` 需要知道它应该操作哪个陷阱帧。通过 `move a0, sp`，内核将陷阱帧的地址放入 `a0`，C 函数 `trap(struct trapframe *tf)` 就可以通过 `a0` 寄存器获取这个地址，并以结构体指针 `tf` 的形式来访问和修改被中断进程的状态。

SAVE_ALL中寄存器保存在栈中的位置是什么确定的？

确定方式：由内核开发人员在SAVE_ALL宏的汇编代码中显式定义和严格排序，并与C语言定义的陷阱帧结构体保持一致。

代码块

```
1  .macro SAVE_ALL
2
3      csrw sscratch, sp
4
5      addi sp, sp, -36 * REGBYTES
6      # save x registers
7      STORE x0, 0*REGBYTES(sp)
8      STORE x1, 1*REGBYTES(sp)
9      STORE x3, 3*REGBYTES(sp)
10     STORE x4, 4*REGBYTES(sp)
11     STORE x5, 5*REGBYTES(sp)
12     STORE x6, 6*REGBYTES(sp)
13     STORE x7, 7*REGBYTES(sp)
14     STORE x8, 8*REGBYTES(sp)
15     STORE x9, 9*REGBYTES(sp)
16     STORE x10, 10*REGBYTES(sp)
17     STORE x11, 11*REGBYTES(sp)
18     STORE x12, 12*REGBYTES(sp)
19     STORE x13, 13*REGBYTES(sp)
20     STORE x14, 14*REGBYTES(sp)
21     STORE x15, 15*REGBYTES(sp)
22     STORE x16, 16*REGBYTES(sp)
23     STORE x17, 17*REGBYTES(sp)
24     STORE x18, 18*REGBYTES(sp)
25     STORE x19, 19*REGBYTES(sp)
26     STORE x20, 20*REGBYTES(sp)
27     STORE x21, 21*REGBYTES(sp)
28     STORE x22, 22*REGBYTES(sp)
29     STORE x23, 23*REGBYTES(sp)
30     STORE x24, 24*REGBYTES(sp)
31     STORE x25, 25*REGBYTES(sp)
32     STORE x26, 26*REGBYTES(sp)
33     STORE x27, 27*REGBYTES(sp)
34     STORE x28, 28*REGBYTES(sp)
35     STORE x29, 29*REGBYTES(sp)
```

```

36     STORE x30, 30*REGBYTES(sp)
37     STORE x31, 31*REGBYTES(sp)
38
39     # get sr, epc, badvaddr, cause
40     # Set sscratch register to 0, so that if a recursive exception
41     # occurs, the exception vector knows it came from the kernel
42     csrrw s0, sscratch, x0
43     csrr s1, sstatus
44     csrr s2, sepc
45     csrr s3, sbadaddr
46     csrr s4, scause
47
48     STORE s0, 2*REGBYTES(sp)
49     STORE s1, 32*REGBYTES(sp)
50     STORE s2, 33*REGBYTES(sp)
51     STORE s3, 34*REGBYTES(sp)
52     STORE s4, 35*REGBYTES(sp)
53     .endm

```

也就是上述代码的定义内容。顺序是人为确定的，目的是在C语言中能通过一个清晰的结构体来访问底层的汇编数据。

对于任何中断，__alltraps 中都需要保存所有寄存器吗？

对于任何进入内核的陷阱，__alltraps中都需要保存所有通用寄存器。

理由：

- 通用性：** __alltraps 是所有中断和陷阱的**唯一通用入口点**。内核在进入这个入口时，无法确定发生了哪种类型的事件，以及被中断的代码是否依赖所有寄存器来正确恢复执行。
- 破坏性（C 语言函数调用）：** __alltraps 最终会执行 jal trap 来调用 C 语言函数。
 - C语言的函数调用会遵循**调用约定**，这意味着 C 函数有权**修改**一些寄存器（即**调用者保存**寄存器，如 a0-a7，t0-t6 等）而无需恢复它们。
 - 为了防止 C 函数破坏被中断程序的上下文，内核必须在调用 C 函数**之前**，将所有可能被 C 函数修改的寄存器以及**调用者应该保存**的寄存器（如 ra 等）全部保存起来。
- 进程切换：**在陷阱处理结束时，内核可能决定进行进程切换。如果一个进程被切换出去，其完整的 CPU 状态必须被完整地保存在其进程控制块（PCB）中。如果缺少任何一个寄存器的值，这个进程就无法正确恢复执行。

三、扩展练习 Challenge 2：理解上下文切换机制

在ucore中，上下文切换主要是在trapentry.S文件中进行的。我们通过分析trapentry.S文件来理解上下文切换机制。

该文件主要由两个宏和两个标签构成。

保存上下文

通过 `SAVE_ALL` 宏来保存上下文，`SAVE_ALL` 在陷阱发生时执行，负责将所有必要信息压入栈中，形成陷阱帧。

`cswr sscratch, sp` 将当前栈指针 `sp` 写入 `sscratch` 寄存器，来备份旧的 `sp`。如果陷阱来自用户空间，`sscratch` 在硬件跳转时会和 `sp` 交换值。这里将其视为一个暂存空间，以便后续将其存入陷阱帧，作为被中断代码的 `sp`。

`addi sp, sp, -36 * REGBYTES` 将栈指针下移，为陷阱帧预留36个寄存器大小来确保栈上有足够空间存储所有32个通用寄存器以及4个CSR寄存器，这是创建陷阱帧的第一步。

代码块

```
1      STORE x0, 0*REGBYTES(sp)
2      STORE x1, 1*REGBYTES(sp)
3      STORE x3, 3*REGBYTES(sp)
4      STORE x4, 4*REGBYTES(sp)
5      STORE x5, 5*REGBYTES(sp)
6      STORE x6, 6*REGBYTES(sp)
7      STORE x7, 7*REGBYTES(sp)
8      STORE x8, 8*REGBYTES(sp)
9      STORE x9, 9*REGBYTES(sp)
10     STORE x10, 10*REGBYTES(sp)
11     STORE x11, 11*REGBYTES(sp)
12     STORE x12, 12*REGBYTES(sp)
13     STORE x13, 13*REGBYTES(sp)
14     STORE x14, 14*REGBYTES(sp)
15     STORE x15, 15*REGBYTES(sp)
16     STORE x16, 16*REGBYTES(sp)
17     STORE x17, 17*REGBYTES(sp)
18     STORE x18, 18*REGBYTES(sp)
19     STORE x19, 19*REGBYTES(sp)
20     STORE x20, 20*REGBYTES(sp)
21     STORE x21, 21*REGBYTES(sp)
22     STORE x22, 22*REGBYTES(sp)
23     STORE x23, 23*REGBYTES(sp)
24     STORE x24, 24*REGBYTES(sp)
25     STORE x25, 25*REGBYTES(sp)
26     STORE x26, 26*REGBYTES(sp)
27     STORE x27, 27*REGBYTES(sp)
28     STORE x28, 28*REGBYTES(sp)
29     STORE x29, 29*REGBYTES(sp)
30     STORE x30, 30*REGBYTES(sp)
31     STORE x31, 31*REGBYTES(sp)
```

使用宏STORE将32个通用寄存器一次存储到栈中预留的位置。**保存通用寄存器**。虽然 `x0` (零寄存器) 恒为 0，但此处将其存储是为了保持陷阱帧结构的一致性。

`csrrw s0, sscratch, x0` 将sscratch的值读入临时寄存器s0，同时将0写入sscratch来获取被中断代码的sp，同时清空sscratch，作为内核态的标记，防止后续递归异常。

代码块

```
1      csrr s1, sstatus
2      csrr s2, sepc
3      csrr s3, sbadaddr
4      csrr s4, scause
```

将RISC-V的四个控制状态寄存器的值读入临时寄存器s1到s4从而获取陷阱发生时的关键状态：

`sstatus` (状态寄存器)、`sepc` (异常程序计数器)、`sbadaddr` (错误地址)、`scause` (异常原因)。

`STORE s0, 2*REGBYTES(sp)` 将之前从 `sscratch` 读取的**被中断代码的栈指针** (即 `x2` / `sp` 的值) 存储到栈中。保存 `sp` 寄存器。注意，它存储在偏移量 `2` 处，因为 `x2` 对应于通用寄存器中的 `sp`。

代码块

```
1      STORE s1, 32*REGBYTES(sp)
2      STORE s2, 33*REGBYTES(sp)
3      STORE s3, 34*REGBYTES(sp)
4      STORE s4, 35*REGBYTES(sp)
```

将之前读取的**4个CSR**的值存储到栈中，紧接在 32 个通用寄存器之后。保存关键CSR状态，完成了陷阱帧的构建。

恢复上下文

在陷阱处理完成后，通过 `RESTORE_ALL` 宏来将陷阱帧中的信息恢复到CPU寄存器中。

代码块

```
1      LOAD s1, 32*REGBYTES(sp)
2      LOAD s2, 33*REGBYTES(sp)
```

从栈上加载sstatus和 `sepc` 的值到临时寄存器 `s1` 和 `s2`，来准备恢复最重要的状态。

```
代码块  csrw sstatus, s1
2        csrw sepc, s2
```

将加载的值写回对应的CSR，恢复特权级状态和返回地址。

代码块

```
1        LOAD x1, 1*REGBYTES(sp)
2        LOAD x3, 3*REGBYTES(sp)
3        LOAD x4, 4*REGBYTES(sp)
4        LOAD x5, 5*REGBYTES(sp)
5        LOAD x6, 6*REGBYTES(sp)
6        LOAD x7, 7*REGBYTES(sp)
7        LOAD x8, 8*REGBYTES(sp)
8        LOAD x9, 9*REGBYTES(sp)
9        LOAD x10, 10*REGBYTES(sp)
10       LOAD x11, 11*REGBYTES(sp)
11       LOAD x12, 12*REGBYTES(sp)
12       LOAD x13, 13*REGBYTES(sp)
13       LOAD x14, 14*REGBYTES(sp)
14       LOAD x15, 15*REGBYTES(sp)
15       LOAD x16, 16*REGBYTES(sp)
16       LOAD x17, 17*REGBYTES(sp)
17       LOAD x18, 18*REGBYTES(sp)
18       LOAD x19, 19*REGBYTES(sp)
19       LOAD x20, 20*REGBYTES(sp)
20       LOAD x21, 21*REGBYTES(sp)
21       LOAD x22, 22*REGBYTES(sp)
22       LOAD x23, 23*REGBYTES(sp)
23       LOAD x24, 24*REGBYTES(sp)
24       LOAD x25, 25*REGBYTES(sp)
25       LOAD x26, 26*REGBYTES(sp)
26       LOAD x27, 27*REGBYTES(sp)
27       LOAD x28, 28*REGBYTES(sp)
28       LOAD x29, 29*REGBYTES(sp)
29       LOAD x30, 30*REGBYTES(sp)
30       LOAD x31, 31*REGBYTES(sp)
```

使用宏LOAD按顺序从栈中加载所有通用寄存器的值，恢复除x0(零寄存器)和x2(栈指针sp)之外的所有通用寄存器。

`LOAD x2, 2*REGBYTES(sp)` 最后加载栈指针x2（即sp）的值，恢复栈指针。必须最后恢复sp，否则一旦sp改变，后续的LOAD操作将无法访问到正确的陷阱帧。

上下文切换

上下文切换的完整流程发生在 `__alltraps` 和 `__trapret` 之间，涉及以下步骤：

阶段一：保存当前上下文（`__alltraps` / `SAVE_ALL`）

1. **硬件跳转**：CPU 接收到陷阱信号，硬件自动将执行流切换到内核模式，并跳转到 `__alltraps`。
2. **栈准备**：`SAVE_ALL` 宏调整栈指针 `sp`，为陷阱帧预留空间。
3. **状态保存（`SAVE_ALL`）**：
 - **通用状态**：将所有 32 个通用寄存器（`x0` 到 `x31`）存储到栈上。
 - **特权状态**：读取 `sstatus`，`sepc`，`scause` 等 CSR 并存储到栈上。
 - **结果**：栈上形成一个完整的、不可分割的“陷阱帧”（即被中断进程的上下文）。

阶段二：C 语言处理（`jal trap`）

1. **调用处理函数**：`jal trap` 将陷阱帧的地址作为参数，调用 C 语言的 `trap` 函数。
2. **逻辑判断**：C 函数 `trap` 分析 `scause` 寄存器的值，判断是哪种陷阱（中断、异常或系统调用）。
3. **处理（可能发生切换）**：
 - 对于系统调用或时钟中断，内核可能决定运行时间片结束，需要进行**进程调度（Scheduler）**。
 - 如果需要切换到新进程：调度器会找到下一个要运行的进程，并将其**旧的陷阱帧**（或旧的上下文）保存到该进程的 **PCB** 中，然后将**新进程的陷阱帧**地址设置为返回地址。

阶段三：恢复新上下文（`__trapret` / `RESTORE_ALL`）

1. **恢复特权状态（`RESTORE_ALL`）**：从栈上加载并恢复新进程的 `sstatus` 和 `sepc` 寄存器。
2. **恢复通用状态（`RESTORE_ALL`）**：从栈上加载并恢复新进程的 `x1`，`x3` 到 `x31` 寄存器。
3. **恢复栈（`RESTORE_ALL`）**：最后恢复 `x2`（即 `sp`）。
4. **返回用户/内核空间**：`sret` 指令执行特权级切换，CPU 根据恢复的 `sepc` 跳转到新进程被中断前的执行点，完成了上下文切换。

问题回答

在 `trapentry.S` 中汇编代码 `csrw sscratch, sp; csrrw s0, sscratch, x0` 实现了什么操作，目的是什么？

`csrw sscratch, sp;` 将当前的栈指针 `sp` 的值写入 `sscratch` 寄存器，这是一个**预备性操作**，用于**备份当前的内核栈指针**。如果接下来发生的是一个**内核态陷阱**（即在内核代码执行过程中发生中断），那么这个被备份的 `sp` 就是被中断前的内核栈指针。

`csrrw s0, sscratch, x0` 将 `sscratch` 寄存器中的值读出并写入到通用寄存器 `s0` 中，同时，将零寄存器 `x0` 写入 `sscratch` 寄存器。获取旧的 `sp`，无论陷阱来自用户态还是内核态，`s0` 现在都保

存了被中断前的栈指针。如果为用户态陷阱，硬件在进入 `__alltraps` 之前，会自动交换 `sp` 和 `sscratch` 的值。因此，`sscratch` 中保存的是**用户态的 `sp`**。这条指令将用户 `sp` 读给 `s0`；若为内核态陷阱，硬件不会交换。由于我们之前执行了 `csrw sscratch, sp`，`sscratch` 中保存的是**内核态的 `sp`**。这条指令将内核 `sp` 读给 `s0`。设置内核标记，将 `sscratch` 设置为 `0`，作为一种**约定标记**，表示**当前正在执行陷阱处理程序**。如果在这个状态下再次发生递归异常，内核就知道当前是在陷阱处理流程中。

通过这两条指令，内核成功并安全地获取了被中断代码的栈指针并将其存入 `s0`，然后将 `s0` 存入陷阱帧中，以备将来恢复。

save all里面保存了 `stval` `scause` 这些 `csr`，而在 restore all 里面却不还原它们？那这样 store 的意义何在呢？

- **`stval`**：存储发生异常时的错误地址。
 - **意义**：这是一个**调试信息**或**错误诊断信息**。它只对当前的陷阱处理逻辑（即 `exception_handler`）有意义。
 - **为什么不恢复**：当一个进程恢复执行时，它不需要知道上一个陷阱发生的错误地址是什么。这个寄存器**不属于**进程的可执行上下文。
- **`scause`**：存储发生异常/中断的**原因**。
 - **意义**：这是一个**诊断信息**。它只在陷阱发生后，用于内核识别事件类型。
 - **为什么不恢复**：当进程恢复执行时，恢复旧的 `scause` 没有意义，因为 CPU 可能会很快（甚至在下一条指令）触发新的中断或异常。这个寄存器应该反映**当前**的处理器状态。

STORE 的意义何在？

`SAVE_ALL` 宏存储这些 CSR 的意义在于：

1. **在 C 语言中访问**：在 `SAVE_ALL` 完成后，`stval` 和 `scause` 的值被存储在陷阱帧 (`struct trapframe`) 的特定位置。
2. **传递信息给 C 函数**：C 语言的 `trap` 函数通过 `struct trapframe *tf` 参数，可以直接访问 `tf->badvaddr` 和 `tf->cause` 等字段。
3. **核心处理依据**：C 语言的 `trap_dispatch` 和 `exception_handler` 函数正是依赖这些存储在陷阱帧中的 `scause` 和 `stval` 信息，来决定如何处理这个陷阱。

简而言之：`scause` 和 `stval` 被保存在陷阱帧中，**是为了让 C 语言处理程序能够读取并使用这些诊断信息**。一旦 C 语言函数处理完毕，这些信息便完成了它们的使命，不需要再被写回 CPU 硬件，因为它们对程序未来的执行流程没有影响。

四、扩展练习 Challenge 3：完善异常中断

处理异常的函数是 `exception_handler`，在此做补充：

1. 非法指令异常处理

代码块

```
1  case CAUSE_ILLEGAL_INSTRUCTION:
2      // 非法指令异常处理
3      /* LAB3 CHALLENGE3 YOUR CODE 2313546: */
4      /*(1)输出指令异常类型 (Illegal instruction)
5       *(2)输出异常指令地址
6       *(3)更新 tf->epc 寄存器
7       */
8      cprintf("Exception type: Illegal instruction\n");
9      cprintf("Illegal instruction caught at 0x%08x\n", tf->epc);
10     tf->epc += 4;
11     break;
```

2. 断点异常处理

代码块

```
1  case CAUSE_BREAKPOINT:
2      // 断点异常处理
3      /* LAB3 CHALLENGE3 YOUR CODE 2313546: */
4      /*(1)输出指令异常类型 ( breakpoint)
5       *(2)输出异常指令地址
6       *(3)更新 tf->epc 寄存器
7       */
8      cprintf("Exception type: Breakpoint\n");
9      cprintf("ebreak caught at 0x%08x\n", tf->epc);
10     tf->epc += 4;
11     break;
```

这两个异常的补充十分简单，不对代码做过多的说明。唯一注意的是RISC-V的标准指令长度均为4字节，故 `tf->epc` 需要加4。

五、实验中的重要知识点

1. RISC-V 中的“陷入（Trap）”

在 RISC-V 架构中，**中断（Interrupt）**和**异常（Exception）**统称为 **Trap**。

Trap 是指：CPU 在执行指令过程中，**跳转到特权级的异常/中断处理例程**的一种控制流改变。

区别如下：

类型	触发时机	举例
中断（Interrupt）	异步事件（外部信号）	时钟中断、外部中断、软件中断
异常（Exception）	同步事件（当前指令导致）	非法指令、访问越界、页错误

在硬件上，它们都会触发一次 **Trap 进入过程**，只是来源不同。

2. RISC-V 中断处理的关键控制寄存器（CSR）

CSR寄存器	含义
sstatus	状态寄存器 (保存全局中断使能位、特权级等)
sie	中断使能寄存器 (控制哪些中断可以触发)
stvec	Trap向量表基址 (指定中断/异常处理程序入口)
sepc	保存被中断指令的地址
scause	记录中断或异常的原因 (类型号)
stval	保存相关的错误信息 (比如访问的非法地址)
sscratch	提供一个安全的临时寄存器，在trap入口用于交换或保存sp

- **sstatus寄存器几个关键字段**
 - SIE(Supervisor Interrupt Enable)：是否允许**S模式**中断。为1表示允许中断进入，为0表示禁用中断。（对于U模式无效）
 - SPIE(Supervisor Previous Interrupt Enable)：保存Trap进入前SIE的值（用于 `sret` 返回恢复）。
 - SPP(Supervisor Previous Privilege)：保存Trap前的特权级（0表示U模式，1表示S模式）。

六、其他重要知识点

1. Trap向量表

Trap向量表（Trap Vector Table） 是一张表，存放着“各种中断或异常类型对应的处理程序（函数）的入口地址”的映射关系。也就是说，当系统发生中断或异常时，CPU不需要自己去判断调用哪个函数，而是通过查表自动跳转到对应的处理程序。

RISC-V的 `stvec` 寄存器的最低两位控制其模式：

模式	值	说明
Direct模式	00	Trap都跳转到同一个地址，即 <code>stvec.BASE</code> 。
Vectored模式	01	对于中断，跳转地址 = <code>stvec.BASE + 4 × cause</code> 。异常仍跳到 <code>stvec.BASE</code> 。